

Terraform Cloud to S3 State Migration

Runbook for migrating Terraform state from Terraform Cloud (TFC) to the EPIC S3 backend (pge-epic-tfstate).

Prerequisites

- terraform CLI installed (match the version used by the TFC workspace)
- TFC_TOKEN exported (team or user token with read access to the org)
- AWS creds for the EPIC account (750713712981) with read/write to pge-epic-tfstate and the KMS key (or whatever the official account will be)
- Backup location for pulled state files (local dir or separate S3 prefix)

1. Inventory TFC workspaces

```
curl -s --header "Authorization: Bearer $TFC_TOKEN" \
  "https://app.terraform.io/api/v2/organizations/pgetech/workspaces" \
  | jq -r '.data[].attributes.name' > workspaces.txt
```

Review workspaces.txt and confirm the list matches what you intend to migrate.

2. Export workspace variables (do this before anything else)

TFC variables do not migrate with state. Pull them for each workspace and stash them somewhere safe.

```
WS_ID=$(curl -s --header "Authorization: Bearer $TFC_TOKEN" \
  "https://app.terraform.io/api/v2/organizations/pgetech/workspaces/<workspace-name>" \
  | jq -r '.data.id')
```

```
curl -s --header "Authorization: Bearer $TFC_TOKEN" \
  "https://app.terraform.io/api/v2/workspaces/$WS_ID/vars" \
  | jq '.data[] | {key: .attributes.key, value: .attributes.value, sensitive: .attributes.sensitive, category: .attributes.category}' \
  > vars-<workspace-name>.json
```

Sensitive values come back as null. Track those separately and source them from the original system of record (vault, pipeline variable group, etc).

3. Back up current state

From the repo or directory that owns the workspace:

```
terraform login
terraform init
terraform state pull > backup-<workspace-name>.tfstate
```

Verify the file is valid JSON and non-empty before continuing.

4. Swap the backend block

Remove the `cloud {}` block and replace with the S3 backend. Use a per-workspace key prefix.

```
terraform {  
  backend "s3" {  
    bucket      = "pge-epic-tfstate"  
    key         = "workspaces/<workspace-name>/terraform.tfstate"  
    region      = "us-west-2"  
    encrypt     = true  
    kms_key_id  = "alias/pge-epic-tfstate"  
    use_lockfile = true  
  }  
}
```

5. Migrate state

```
terraform init -migrate-state
```

Terraform detects the backend change and prompts to copy state from TFC to S3. Answer yes.

6. Verify

```
terraform state list  
terraform plan
```

The plan must show **no changes**. If it shows drift, stop and investigate before proceeding. Common causes: provider version mismatch between TFC remote execution and local runner, missing variables, or different AWS credentials in play.

7. Rewire execution

State is now in S3, but TFC was also running plans and applies. Move execution into the EPIC pipeline:

- Wire up AWS OIDC role (pge-epic-deployment-role) on the pipeline stage
- Inject variables from pipeline variable groups (replacing TFC workspace vars)
- Replace any Sentinel/OPA policy enforcement with a CI step (Checkov, tfsec, OPA)

8. Lock TFC workspace (do not delete yet)

Lock the workspace in TFC so nothing runs against the old state while you validate the new pipeline end-to-end.

```
curl -X POST --header "Authorization: Bearer $TFC_TOKEN" \  
      --header "Content-Type: application/vnd.api+json" \  
      --data '{"workspace_id": "workspace-name", "lock": true}'
```

```
"https://app.terraform.io/api/v2/workspaces/$WS_ID/actions/lock" \
-d '{"reason":"Migrated to S3 backend"}'
```

9. Validate in pipeline

Run a real plan and apply through the EPIC pipeline against the S3-backed state. Confirm:

- State lock acquires and releases cleanly
- Plan output matches expectations
- Apply succeeds and updates state in S3

10. Decommission TFC workspace

Only after the pipeline has successfully run apply against the new backend:

```
curl -X DELETE --header "Authorization: Bearer $TFC_TOKEN" \
  "https://app.terraform.io/api/v2/organizations/pgetech/workspaces/<workspace-name>"
```

Run history and audit logs are lost on delete. Export anything needed for compliance first.

Gotchas

- **TFC variables don't migrate.** Handled in step 2, but worth repeating. Sensitive values are write-only in TFC's API.
- **Remote execution moves to the pipeline.** S3 is state-only. Provider auth, env vars, and tooling all need to exist on the runner.
- **Run history stays in TFC** until workspace deletion. Export for audit if required.
- **State locking via S3 conditional writes** (`use_lockfile=true`) is the current standard. DynamoDB locking is no longer required.
- **Sentinel/OPA enforcement disappears.** If TFC was enforcing policy, replace with a CI stage before cutover.
- **TFC-specific features are lost:** no-code modules, run tasks, drift detection, structured run output, workspace tags. Inventory what you depend on before cutting over.
- **Provider version drift.** TFC pins providers per workspace. Make sure the pipeline runner uses the same versions or you'll see spurious diffs.

Bulk migration

For many workspaces, script the loop:

1. Read `workspaces.txt`
2. For each workspace: export vars, pull state backup, rewrite backend block, run `init -migrate-state -force-copy`, run `plan`, log result
3. Halt on any non-zero plan diff for manual review

The rewrite step depends on how the backend block is structured across repos. If every repo uses an identical `cloud {}` block, a `sed` swap works. If they vary, use `hcledit` or a templated approach.

Rollback

If something goes wrong after migration but before TFC workspace deletion:

1. Restore the original backend block (`cloud {}`)
2. `terraform init -migrate-state` will prompt to copy state back from S3 to TFC
3. Unlock the TFC workspace

If the TFC workspace is already deleted, restore from `backup-<workspace-name>.tfstate` by pushing it to a new workspace or back into S3.